

1. Теорія: що таке Git і як він влаштований

Навіщо потрібен Git

Уяви, що над одним кодом одночасно працює кілька людей. Кожен додає зміни, щось ламає, щось виправляє. Без системи контролю версій хаос неминучий.

Знайомі симптоми життя без Git:

- папки `project`, `project_final`, `project_final_2`, `project_final_ПРАВКИ`;
- «а в кого остання версія?»;
- зламав робочий код і не пам'ятаєш, як було до цього.

Git — це «машина часу» для коду. Він дозволяє:

- зберігати історію змін (усі версії коду, а не лише останню);
- працювати командою над одним проєктом;
- безпечно експериментувати в гілках;
- швидко «відкотитись», якщо щось зламалось.

Git vs GitHub vs GitLab

Інструмент	Що це	Для чого
Git	система контролю версій (локально на комп'ютері)	відстежує зміни у файлах
GitHub	онлайн-платформа	зберігає репозиторії у «хмарі», дозволяє співпрацювати
GitLab	альтернатива GitHub (створена українцями)	більше фіч для DevOps і CI/CD

Аналогія:

- **Git** — це «двигун».
- **GitHub** — це «гараж, де цей двигун стоїть і де ми збираємось».

Ключове: Git працює **без інтернету і без жодного сайту**. GitHub / GitLab / Bitbucket — лише місця, де копію репозиторію зручно тримати спільно. Тому команди `git add`, `git commit` працюють завжди, а `git push` / `git pull` — лише коли є віддалений репозиторій.

Коротка історія

- **2005 рік** — Git створив **Лінус Торвальдс**, коли розробляв ядро Linux.
- Написав систему приблизно **за два тижні**, бо попередній інструмент (BitKeeper) став платним.
- Сьогодні Git — стандарт у світі розробки, ним користуються практично всі.
- Історія зберігається як **ланцюжок**: кожен коміт пов'язаний хешем із попереднім — принцип той самий, що в блокчейні. Тому історію не можна «тихо» підправити: зміниться один коміт — зміняться хеші всіх наступних.

Основна ідея: три зони Git

У Git три «зони», і майже кожна команда — це переміщення змін між ними:

1. **Working directory** (робоча тека) — твої поточні файли, чернетка.
2. **Staging area** (index, «сцена») — підготовлені зміни, які потраплять у коміт.
3. **Repository** (`.git`) — історія всіх комітів.

```
правка файлу      git add      git commit      git push
Working directory → Staging area → Repository → GitHub / GitLab
(чернетка)         (кошик)      (.git)           (хмара)
```

Навіщо взагалі потрібна staging area? Щоб зібрати **осмислений** коміт. Ти змінив п'ять файлів, але три з них стосуються однієї задачі, а два — іншої. `git add` дає можливість покласти в коміт лише потрібне.

Зворотний напрямок:

Куди повертаємо	Команда
зі staging назад у working directory	<code>git restore --staged <file></code>
скасувати правки у working directory	<code>git restore <file></code>

Інтеграція в IDE

PyCharm

- Git інтегрований за замовчуванням;
- можна комітити, пушити, створювати PR прямо з IDE;
- є графічна історія комітів (**Git** → **Log**);
- сполучення: `Ctrl+K` — коміт, `Ctrl+Shift+K` — пуш.

VS Code

- панель **Source Control** (`Ctrl+Shift+G`);
- показує змінені файли й різницю по рядках;
- можна комітити, пушити, створювати гілки;
- корисні розширення: **GitLens**, **Git Graph**.

Порада на початку навчання: роби перші 20–30 комітів **руками в терміналі**. Кнопки в IDE приховують те, що насправді відбувається, і потім важко розбиратися, коли щось піде не так.